UNIT II: Functions and Classes & Objects in C++

Section A: Debugging Questions

Question 1: Dangling Reference in Return-by-Reference

Topic: Return by Reference, Scope, Activation Records

Corrected Code:

```
#include <iostream>
using namespace std;
int getMaximumScaled(int a, int b, int scaleFactor) {
    int maxVal = (a > b) ? a : b;
    int result = maxVal * scaleFactor;
    return result;
}

int& getArrayElement(int arr[], int index) {
    return arr[index];
}

int main() {
    int x = 10, y = 20;

    int scaledValue = getMaximumScaled(x, y, 2);

    cout << "Scaled Max: " << scaledValue << endl;

    int data[3] = {100, 200, 300};

    getArrayElement(data, 1) = 999;

    cout << "Updated Array[1]: "
         << data[1] << endl;

    cout << "Scaled Value: "
         << scaledValue << endl;
```

```
    return 0;
}
```

Output:

Scaled Max: 40

Updated Array[1]: 999

Scaled Value: 40

Question 2: Static Data Member Linkage & Member Function Scoping

Topic: Static Data Members, Static Member Functions, Scope Resolution

Corrected Code:

```
#include <iostream>
#include <string>
#include <iomanip>
using namespace std;

class BankAccount {
private:
    string accHolder;
    double balance;

    static double totalBankReserves;

public:
    BankAccount(string name, double initialDeposit) {
        accHolder = name;
        balance = initialDeposit;
        totalBankReserves += initialDeposit;
    }

    void displayAccount() const {
        cout << "Account Holder: " << accHolder << endl;
        cout << "Account Balance: $"
             << balance << endl;
    }

    static void displayAudit() {
        cout << "Total Reserves in Vault: $"
```

```
        << totalBankReserves << endl;
    }
};

// Definition of static data member
double BankAccount::totalBankReserves = 0.0;

int main() {
    cout << fixed << setprecision(2);

    BankAccount acc1("Alice", 5000.0);
    BankAccount acc2("Bob", 3000.0);

    acc1.displayAccount();
    acc2.displayAccount();

    BankAccount::displayAudit();

    return 0;
}
```

Output: Account Holder: Alice
Account Balance: $5000.00
Account Holder: Bob
Account Balance: $3000.00
Total Reserves in Vault: $8000.00

Question 3: Function Overload Ambiguity via Default Arguments

Topic: Function Overloading, Default Arguments

Corrected Code:

```
#include <iostream>
using namespace std;

// Cuboid volume
double computeCuboidVolume(double length,
                           double width = 1.0,
                           double height = 1.0) {
    return length * width * height;
}
```

```
// Cube volume
double computeCubeVolume(double side) {
    return side * side * side;
}

// Cylinder volume
double computeCylinderVolume(double radius,
                             double height) {
    return 3.14159 * radius * radius * height;
}

int main() {
    cout << "Cube Volume: "
         << computeCubeVolume(5.0) << endl;

    cout << "Cuboid Volume: "
         << computeCuboidVolume(5.0) << endl;

    cout << "Cylinder Volume: "
         << computeCylinderVolume(3.0, 7.0) << endl;

    return 0;
}
```

Output: Cube Volume: 125

Cuboid Volume: 5

Cylinder Volume: 197.92

Question 4: Array of Objects Missing Default Constructor

Topic: Arrays of Objects, Constructors, Class Specification

Corrected Code:

```
#include <iostream>
#include <string>
using namespace std;

class Student {
```

```
private:
    int rollNumber;
    string name;

public:
    // Default constructor
    Student() {
        rollNumber = 0;
        name = "";
    }

    // Parameterized constructor
    Student(int r, string n) {
        rollNumber = r;
        name = n;
    }

    void display() const {
        cout << "Roll: " << rollNumber
             << ", Name: " << name << endl;
    }
};

int main() {
    Student batch[3];

    batch[0] = Student(101, "Aarav");
    batch[1] = Student(102, "Bhavna");
    batch[2] = Student(103, "Chirag");

    cout << "Student Records:" << endl;

    for (int i = 0; i < 3; ++i) {
        batch[i].display();
    }
```

```
    return 0;
}
```

Output:

```
Student Records:
Roll: 101, Name: Aarav
Roll: 102, Name: Bhavna
Roll: 103, Name: Chirag
```

Question 5: Friend Function Declaration & Scope Resolution Flaws

Topic: Friendly Functions, Forward Declarations, Member vs. Non-Member

Corrected Code:

```
#include <iostream>
using namespace std;

// Forward declarations
class Engine;
class TelemetryAuditor;

// Friend function prototype
void inspectDiagnostics(Engine e, TelemetryAuditor a);

class Engine {
private:
    int temperature;
    int rpm;

public:
    Engine(int t, int r)
        : temperature(t), rpm(r) {}

    friend void inspectDiagnostics(
        Engine e, TelemetryAuditor a
    );
};
```

```
class TelemetryAuditor {
private:
    int alertThreshold;

public:
    TelemetryAuditor(int threshold)
        : alertThreshold(threshold) {}

    friend void inspectDiagnostics(
        Engine e, TelemetryAuditor a
    );
};

// Friend function definition
void inspectDiagnostics(Engine e, TelemetryAuditor a) {
    if (e.temperature > a.alertThreshold) {
        cout << "Warning: Engine overheat!" << endl;
    }
    else {
        cout << "Engine operating nominally at "
             << e.rpm << " RPM." << endl;
    }
}

int main() {
    Engine v8(105, 3200);
    TelemetryAuditor audit(100);

    // Friend function called like a normal function
    inspectDiagnostics(v8, audit);

    return 0;
}
```

Output:

```
Warning: Engine overheat!
```

Section B: Pseudo-Code Writing Questions

Question 1: Corporate Vault Management with Static Data & Functions

Topic: Static Data Members, Static Member Functions, Encapsulation

pseudo-code:

```
CLASS BranchOffice
PRIVATE:
    branchID : STRING
    branchName : STRING
    currentBalance : REAL
    STATIC totalVaultReserves : REAL ← 0.0
    STATIC totalBranchesActive : INTEGER ← 0
PUBLIC:
    CONSTRUCTOR Initialize(id, name, openingDeposit)
        branchID ← id
        branchName ← name
        currentBalance ← openingDeposi
        totalVaultReserves ←
            totalVaultReserves + openingDeposit
        totalBranchesActive ←
            totalBranchesActive + 1
    END CONSTRUCTOR
    METHOD Deposit(amount)
        IF amount > 0 THEN
            currentBalance ← currentBalance + amount
            totalVaultReserves ←
                totalVaultReserves + amount
            DISPLAY "Deposit Successful"
        ELSE
            DISPLAY "Invalid Deposit Amount"
        ENDIF
    END METHOD
    METHOD Withdraw(amount) RETURNS BOOLEAN
        IF amount > 0 AND amount <= currentBalance THEN
            currentBalance ← currentBalance - amount
            totalVaultReserves ←
                totalVaultReserves - amount
            RETURN TRUE
        ELSE
            RETURN FALSE
        ENDIF
```

```
    END METHOD
    STATIC METHOD GetVaultSummary()
        DISPLAY "Active Branches: ",
                totalBranchesActive
        DISPLAY "Total Vault Reserves: ",
                totalVaultReserves
    END STATIC METHOD
    STATIC METHOD GetGlobalLiquidityRatio(targetReserve)
        IF targetReserve <= 0 THEN
            RETURN 0
        ENDIF
        RETURN totalVaultReserves / targetReserve
    END STATIC METHOD
ENDCLASS
BEGIN MAIN
    DECLARE Branch1, Branch2 : BranchOffice
    Branch1.Initialize("B001", "Chennai", 500000)
    Branch2.Initialize("B002", "Tambaram", 300000)
    DISPLAY "Initial Vault Summary"
    BranchOffice.GetVaultSummary()
    DISPLAY "Branch 1 Deposits 150000"
    Branch1.Deposit(150000)
    DISPLAY "Branch 2 Withdraws 50000"
    IF Branch2.Withdraw(50000) = TRUE THEN
        DISPLAY "Withdrawal Successful"
    ELSE
        DISPLAY "Insufficient Balance"
    ENDIF
    DISPLAY "Final Vault Summary"
    BranchOffice.GetVaultSummary()
    DECLARE ratio : REAL
    ratio ← BranchOffice.GetGlobalLiquidityRatio(1000000)
    DISPLAY "Global Liquidity Ratio: ", ratio
END MAIN
```

output:

```
Initial Vault Summary
Active Branches: 2
Total Vault Reserves: 800000
Branch 1 Deposits 150000
Deposit Successful
Branch 2 Withdraws 50000
```

```
Withdrawal Successful
Final Vault Summary
Active Branches: 2
Total Vault Reserves: 900000
Global Liquidity Ratio: 0.9
```

Question 2: Matrix Mathematical Engine with Arrays in Class & Friend Dot-Product

Topic: Arrays within a Class, Scope Resolution `::`, Friend Functions

pseudo-code:

```
CLASS Matrix2D
PRIVATE:
    grid : ARRAY[3][3] OF REAL
PUBLIC:
    DECLARE METHOD Initialize()
    DECLARE METHOD SetElement(row, col, value)
    DECLARE METHOD GetElement(row, col) RETURNS REAL
    DECLARE METHOD Display()
    FRIEND FUNCTION ComputeCrossTrace(
        Matrix2D A, Matrix2D B
    ) RETURNS REAL
ENDCLASS
METHOD Matrix2D::Initialize()
    FOR i ← 0 TO 2
        FOR j ← 0 TO 2
            grid[i][j] ← 0
        ENDFOR
    ENDFOR
END METHOD
METHOD Matrix2D::SetElement(row, col, value)
    IF row >= 0 AND row < 3 AND
       col >= 0 AND col < 3 THEN
        grid[row][col] ← value
    ELSE
        DISPLAY "Invalid Matrix Index"
    ENDIF
END METHOD
METHOD Matrix2D::GetElement(row, col) RETURNS REAL
    IF row >= 0 AND row < 3 AND
       col >= 0 AND col < 3 THEN
        RETURN grid[row][col]
    ELSE
```

```
        DISPLAY "Invalid Matrix Index"
        RETURN 0
    ENDIF
END METHOD
METHOD Matrix2D::Display()
    FOR i ← 0 TO 2
        FOR j ← 0 TO 2
            DISPLAY grid[i][j]
        ENDFOR
        DISPLAY NEWLINE
    ENDFOR
END METHOD
FRIEND FUNCTION ComputeCrossTrace(A, B) RETURNS REAL
    DECLARE sum : REAL
    sum ← 0
    FOR i ← 0 TO 2
        sum ← sum + (A.grid[i][i] * B.grid[i][i])
    ENDFOR
    RETURN sum
END FUNCTION
BEGIN MAIN
    DECLARE A, B : Matrix2D
    A.Initialize()
    B.Initialize()
    A.SetElement(0, 0, 2)
    A.SetElement(1, 1, 4)
    A.SetElement(2, 2, 6)
    B.SetElement(0, 0, 1)
    B.SetElement(1, 1, 3)
    B.SetElement(2, 2, 5)
    DISPLAY "Matrix A"
    A.Display()
    DISPLAY "Matrix B"
    B.Display()
    DECLARE result : REAL
    result ← ComputeCrossTrace(A, B)
    DISPLAY "Cross Trace: ", result
END MAIN
```

output:

```
Matrix A
2 0 0
0 4 0
```

```
0 0 6
Matrix B
1 0 0
0 3 0
0 0 5
Cross Trace: 44
```

Question 3: Student Academic Roster via Array of Objects

Topic: Arrays of Objects, Member Functions, Sorting Objects

pseudo-code:

```
CLASS StudentRecord
PRIVATE:
    rollNo : INTEGER
    studentName : STRING
    marks : ARRAY[3] OF REAL
    percentage : REAL
    grade : CHARACTER
    METHOD CalculateResult()
        percentage ←
            (marks[0] + marks[1] + marks[2]) / 3
        IF percentage >= 85 THEN
            grade ← 'A'
        ELSE IF percentage >= 70 THEN
            grade ← 'B'
        ELSE IF percentage >= 50 THEN
            grade ← 'C'
        ELSE
            grade ← 'F'
        ENDIF
    END METHOD
PUBLIC:
    CONSTRUCTOR Initialize()
        rollNo ← 0
        studentName ← ""
        marks[0] ← 0
        marks[1] ← 0
        marks[2] ← 0
        percentage ← 0
        grade ← 'F'
    END CONSTRUCTOR
    METHOD ReadData(r, name, m1, m2, m3)
```

```
        rollNo ← r
        studentName ← name
        marks[0] ← m1
        marks[1] ← m2
        marks[2] ← m3
        CALL CalculateResult()
    END METHOD
    METHOD GetPercentage() RETURNS REAL
        RETURN percentage
    END METHOD
    METHOD Display()
        DISPLAY "Roll Number: ", rollNo
        DISPLAY "Student Name: ", studentName
        DISPLAY "Percentage: ", percentage
        DISPLAY "Grade: ", grade
    END METHOD
ENDCLASS
BEGIN MAIN
    DECLARE students : ARRAY[3] OF StudentRecord
    DECLARE i, j : INTEGER
    DECLARE temp : StudentRecord
    students[0].ReadData(101, "Arun", 90, 85, 95)
    students[1].ReadData(102, "Bala", 70, 75, 80)
    students[2].ReadData(103, "Charan", 88, 92, 90)
    // Bubble Sort in descending percentage order
    FOR i ← 0 TO 1
        FOR j ← 0 TO 1 - i
            IF students[j].GetPercentage() <
               students[j + 1].GetPercentage() THEN
                temp ← students[j]
                students[j] ← students[j + 1]
                students[j + 1] ← temp
            ENDIF
        ENDFOR
    ENDFOR
    DISPLAY "Student Rank List"
    FOR i ← 0 TO 2
        DISPLAY "Rank: ", i + 1
        students[i].Display()
    ENDFOR
END MAIN
```

output:

```
Student Rank List
Rank: 1
Roll Number: 101
Student Name: Arun
Percentage: 90
Grade: A
Rank: 2
Roll Number: 103
Student Name: Charan
Percentage: 90
Grade: A
Rank: 3
Roll Number: 102
Student Name: Bala
Percentage: 75
Grade: B
```

Question 4: Flexible Logger with Overloaded Functions & Inline Helpers

Topic: Function Overloading, Default Arguments, Inline Functions

pseudo-code:

```
INLINE FUNCTION FormatPrefix(prefixTag : STRING)
    RETURNS STRING
    RETURN "[" + prefixTag + "] "
END FUNCTION
// Overload 1: General information logging
FUNCTION LogEvent(message : STRING)
    DECLARE prefix : STRING
    prefix ← FormatPrefix("INFO")
    DISPLAY prefix + message
END FUNCTION
// Overload 2: Severity-based logging
// Default severity = 1
// Default timestamp = "00:00:00"
FUNCTION LogEvent(
    message : STRING,
    severityLevel : INTEGER = 1,
    timestamp : STRING = "00:00:00"
)
    DECLARE severity : STRING
   SWITCH severityLevel
        CASE 1:
```

```
            severity ← "LOW"
            BREAK
        CASE 2:
            severity ← "MEDIUM"
            BREAK
        CASE 3:
            severity ← "CRITICAL"
            BREAK
        DEFAULT:
            severity ← "UNKNOWN"
    ENDSWITCH
    DISPLAY "[" + severity + "] @ " +
            timestamp + " : " + message
END FUNCTION
// Overload 3: Hardware fault logging
FUNCTION LogEvent(
    errorCode : INTEGER,
    systemModule : STRING
)
    DISPLAY "[HARDWARE_FAULT] Module: " +
            systemModule +
            " -> Error Code: " +
            HEX(errorCode)
END FUNCTION
BEGIN MAIN
    // One argument
    LogEvent("System Started")
    // Two arguments
    LogEvent("Temperature High", 2)
    // Three arguments
    LogEvent("Sensor Failure", 3, "10:30:45")
    // Hardware fault overload
    LogEvent(255, "Engine")
END MAIN
```

output:

```
[INFO] System Started
[MEDIUM] @ 00:00:00 : Temperature High
[CRITICAL] @ 10:30:45 : Sensor Failure
[HARDWARE_FAULT] Module: Engine -> Error Code: 0xFF
```

Question 5: Hardware Register Bank with Return-by-Reference

Topic: Return by Reference, Arrays within a Class, Lvalue Semantics

pseudo-code:

```
CLASS RegisterBank
PRIVATE:
    registers : ARRAY[8] OF INTEGER
    errorSink : INTEGER
PUBLIC:
    CONSTRUCTOR Initialize()
        FOR i ← 0 TO 7
            registers[i] ← 0
        ENDFOR
        errorSink ← -1
    END CONSTRUCTOR
    METHOD GetRegister(index : INTEGER)
        RETURNS REF INTEGER
        IF index >= 0 AND index < 8 THEN
            RETURN REF registers[index]
        ELSE
            DISPLAY "ALERT: Invalid Register Index"
            RETURN REF errorSink
        ENDIF
    END METHOD
    METHOD DisplayState()
        DISPLAY "Register Bank State"
        FOR i ← 0 TO 7
            DISPLAY "Register[", i, "] = ",
                    HEX(registers[i])
        ENDFOR
    END METHOD
ENDCLASS
BEGIN MAIN
    DECLARE bank : RegisterBank
    DECLARE readVal : INTEGER
    bank.Initialize()
    // Direct assignment using returned reference
    bank.GetRegister(0) ← 170
    bank.GetRegister(1) ← 255
    // Read value through returned reference
    readVal ← bank.GetRegister(0)
    DISPLAY "Read Register 0: ", HEX(readVal)
    // Out-of-bounds assignment
    bank.GetRegister(10) ← 999
```

```
    DISPLAY "Error Sink Value: ", bank.GetRegister(10)
    bank.DisplayState()
END MAIN
```

output:

```
    Read Register 0: 0xAA
    ALERT: Invalid Register Index
    ALERT: Invalid Register Index
    Error Sink Value: 999
    Register Bank State
    Register[0] = 0xAA
    Register[1] = 0xFF
    Register[2] = 0x0
    Register[3] = 0x0
    Register[4] = 0x0
    Register[5] = 0x0
    Register[6] = 0x0
    Register[7] = 0x0
```